

AlgeMate: 你的线性代数学习伙伴

大作业报告：鲍欣平 任喜悦 陈泓宇

一、程序功能介绍

主要功能	子功能	功能介绍
AI 智能解题	多模态对话	支持文字/图片输入，调用 DeepSeek API 进行数学题目解答，流式输出回复，支持数学公式 Latex 渲染
AI 智能解题	OCR 识别	上传图片自动 OCR 提取题目文字，填入输入框
AI 智能解题	对话历史	全部对话持久化存储，支持清空重新开始
交互式计算器	表达式求值	逐单元格输入矩阵/多项式/运算表达式，实时计算并渲染 LaTeX 结果
交互式计算器	分步展示	矩阵运算（求逆、LU 分解、特征值等）自动生成分步推导过程
交互式计算器	历史记录	支持回看、编辑历史计算记录
算法演示	11 个专题演示	涵盖特征值、Jordan 标准形、SVD、正交对角化、二次型、线性方程组等
算法演示	可交互输入	用户输入自定义矩阵/多项式参数，即时计算并展示过程
算法演示	公式渲染	全部数学公式通过 LaTeX 渲染为矢量图形
3D 可视化	二次曲面绘制	输入二次型矩阵，实时渲染 3D 二次曲面，支持旋转/缩放
学习中心	知识点	10 章高等代数知识点，Markdown 渲染，支持导航目录
学习中心	练习模式	按章节分类的练习题，支持选择/填空/解答多种题型
学习中心	考试模式	收录往年真题、倒计时、自动判分、成绩统计
学习中心	错题本	自动收录错题，支持回顾重做
学习管理中心	学习时长趋势	分析、查看用户总学习时长的变化趋势
学习管理中心	打卡日历	以学习打卡的方式，增加使用此软件学习的趣味
学习管理中心	时长模块分布	以扇形图的形式，直观清晰展示出用户学习在不同模块的分布
首页	学习目标管理	添加/编辑/完成学习目标，进度可视化
首页	学习记录	按时间统计学习时长和做题数量
设置中心	API 配置	配置 DeepSeek 和豆包 OCR 密钥
设置中心	快捷键	自定义 AI 助手、错题本等功能的全局快捷键
设置中心	个人资料	修改用户名和头像

二、项目各模块与类设计细节

（一）核心架构类

1. MainWindow：主窗口类 `src/ui/MainWindow.h`

成员变量	说明
<code>titleBar</code> (TitleBar*)	顶部标题栏，含 Logo、应用名、字号调节、用户按钮
<code>navBar</code> (NavigationBar*)	左侧导航栏，图标+文字导航项
<code>pageStack</code> (QStackedWidget*)	中央页面栈，通过导航切换不同功能页面
成员方法	说明
<code>composeLayout()</code>	组装标题栏-导航栏-页面栈的整体布局
<code>registerModules()</code>	注册全部功能页面：首页、计算器、AI解题、学习中心、设置

2. NavigationBar：导航栏类 `src/ui/NavigationBar.h`

成员变量	说明
<code>navList</code> (QListWidget*)	导航项列表，每项含图标+文字
成员方法	说明
<code>addNavItem</code> (icon, title)	添加导航项
<code>setCurrentIndex(int)</code> / <code>currentIndex()</code>	获取/设置当前选中索引

3. ThemeManager：主题管理单例 `src/core/ThemeManager.h`

成员变量	说明
<code>m_currentTheme</code> (Theme)	当前主题枚举：Light / Dark
<code>m_fontScale</code> (double)	全局字体缩放系数
成员方法	说明
<code>applyTheme</code> (Theme)	加载对应 QSS 样式表，应用到 QApplication 全局
<code>setFontScale(double)</code> / <code>increaseFont()</code> / <code>decreaseFont()</code> / <code>resetFont()</code>	字体缩放控制

4. UserProfile：用户信息单例 `src/core/UserProfile.h`

成员变量	说明
<code>m_userName</code> (QString)	用户名
<code>m_avatarPath</code> (QString)	头像图片路径
成员方法	说明
<code>avatarPixmap</code> (int size)	返回缩放后的头像 QPixmap
<code>setUserName</code> / <code>setAvatarPath</code>	修改用户信息并持久化
<code>greetingByTime()</code> [static]	根据系统时间返回"早上好/下午好/晚上好"

(二) 首页模块 `src/modules/home/`

HomePage：首页类

成员变量	说明
<code>goalFrame</code> (QFrame*)	学习目标卡片，显示目标标题与进度条
<code>progressPanel</code> (QFrame*)	今日学习统计面板
<code>studyHistoryPanel</code> (QFrame*)	近一周学习时长柱状图区域
成员方法	说明
<code>setupUi()</code>	初始化问候语、目标卡片、进度面板、历史面板布局
<code>addGoal()</code> / <code>editGoal()</code> / <code>deleteGoal()</code>	学习目标的增删改
<code>refreshProgress()</code>	刷新今日学习时长和做题数量
<code>applyTheme()</code>	响应主题切换，更新全部组件颜色

(三) 计算器模块 `src/modules/calculator/`

1. CalculatorPage：计算器模块主页

成员方法	说明
<code>setupUi()</code>	初始化选项卡：交互式计算 / 算法演示 / 3D可视化

成员方法	说明
选项卡切换	三个子页面：InteractivePage、DemoPage、VisualizePage

成员变量	说明
<code>cells_</code> (QVector<HistoryCell*>)	计算历史记录列表
<code>evaluator_</code> (Evaluator)	表达式求值引擎
<code>theme_</code> (RenderTheme)	当前渲染配色

成员方法	说明
<code>onInputSubmitted(input)</code>	接收用户输入，调用 Evaluator 计算，追加新 Cell 并渲染
<code>redrawHistory()</code>	重建全部历史 Cell 的 LaTeX 渲染
<code>applyHistoryPalette()</code>	根据主题刷新调色板

3. Evaluator：表达式求值引擎 `src/modules/calculator/interactive/expr/`

成员变量	说明
<code>m_varTable</code> (map)	变量名 → Value 映射表，支持变量赋值与引用
<code>m_algorithms_</code> (map)	内置算法注册表 (inv, det, lu, eigen, svd, rref...)

成员方法	说明
<code>addVar</code> (name, value)	注册/更新变量
<code>evaluate</code> (text)	主入口：解析输入文本 → 计算 → 返回 Value 结果
<code>callAlgorithm</code> (name, args)	调用内置矩阵算法，返回分步计算过程

协作类说明：

- `Lexer`：词法分析，将输入字符串分解为 Token 流
- `Parser`：语法分析，递归下降解析器构建 AST
- `Value`：值类型联合体（标量 / 矩阵 / 多项式 / 变量引用）
- `RenderSettings`：内含 `RenderTheme` 结构体，定义 LaTeX 渲染配色

4. DemoPage + 11 个演示子页 `src/modules/calculator/demo/`

子页面类	对应算法
EigenPage	实特征值与特征向量
JordanFormPage	Jordan 标准形
GSOPage	Gram-Schmidt 正交化
SymDiagPage	实对称矩阵正交对角化
QuadFormPage	二次型标准化
HomoLinearSystemPage	齐次线性方程组求解
NonhomoLinearSystemPage	非齐次线性方程组求解
InversePage	矩阵求逆
MaxIndepPage	极大无关组
PolyGCDPage	多项式最大公因式
SymReducePage	对称矩阵合同化简

页面逻辑：用户输入矩阵/多项式参数 → 调用 `math/algorithm/` 中对应算法 → LaTeX 渲染完整计算过程。

5. VisualizePage + QuadricWidget：3D 可视化

成员变量	说明
<code>quadricWidget</code> (<code>QuadricWidget*</code>)	OpenGL 二次曲面渲染控件
成员方法	说明
<code>onMatrixInput()</code>	接收二次型矩阵，传递给 <code>QuadricWidget</code> 重绘三维图像
<code>QuadricWidget</code> ：基于 Qt OpenGLWidget 实现，根据二次型矩阵实时绘制椭球面、双曲面、抛物面等三维曲面，支持鼠标拖拽旋转、滚轮缩放。	

(四) AI 解题模块 `src/modules/ai_solver/`

1. AiSolverPage: AI 解题模块主页

成员变量	说明
<code>resultEdit_</code> (<code>LatexTextBrowser*</code>)	LaTeX 文本浏览器，用于渲染 Markdown 与数学公式
<code>inputEdit_</code> (<code>QTextEdit*</code>)	用户输入框，支持多行输入与图片拖拽
<code>statusLabel_</code> (<code>QLabel*</code>)	显示当前网络请求状态、OCR 进度及深度思考模式提示
<code>deepThinkButton_</code> (<code>QPushButton*</code>)	深度思考开关，控制调用 DeepSeek-Reasoner 模型
<code>chatHistory_</code> (<code>QJsonArray</code>)	存储上下文（包括系统提示、用户输入和 AI 回复）
成员方法	说明
<code>mountImagePath()</code>	将用户上传/拖拽的图片挂载到界面上，生成带缩略图的预览卡片
<code>onSendButtonClicked()</code>	触发送，处理图片 OCR 流程或直接调用 DeepSeek API
<code>onDoubaoOcrFinished()</code>	图片识别成功后，自动填充文本并触发 AI 解答逻辑
<code>onReplyReadyRead()</code>	处理流式响应
<code>onReplyFinished()</code>	完成响应，进行 LaTeX 重新渲染与对话历史保存
<code>onClearHistoryClicked()</code>	清空本地缓存的对话记录与界面内容

说明(实现细节):

- 用户输入处理**：可输入文字，也可拖拽/上传图片。点击发送，`onSendButtonClicked()` 完成 API_Keys 校验和内容展示。
- OCR**：若存在图片，先调用豆包 OCR API 提取图片中的数学公式与文字。图片缩放至 1024x1024 并转为 Base64 嵌入请求体。OCR 完成后，识别结果与用户输入拼接，自动转入 DeepSeek 流程。
- DeepSeek**：构建 HTTP POST 请求至 `https://api.deepseek.com/chat/completions`。请求体包含模型（常规 `deepseek-chat` / 深度思考 `deepseek-reasoner`）、完整 `chatHistory_`，以及 `"stream": true` 启用流式响应。
- 上下文管理**：`chatHistory_` 存储完整对话历史，保证多轮对话的上下文连续性。system prompt 在构造时写入，为 AI 设定数学助手角色和 LaTeX 输出规范。
- 流式处理**：SSE 流式响应 (多次触发 `readyRead`):

```
data: {"choices":[{"delta":{"content":"你"}}]}
data: {"choices":[{"delta":{"content":"好"}}]}
data: {"choices":[{"delta":{"content":"! "}}]}
data: [DONE]
```

`onReplyReadyRead()` 每次被调用时追加一个字符,界面显示: "你" → "你好" → "你好! " (打字机效果)

2. OcrAttachWidget: OCR 附件组件

成员变量	说明
<code>uploadBtn_</code> (QPushButton*)	图片上传按钮
<code>clearBtn_</code> (QPushButton*)	清除图片按钮
<code>thumbLabel_</code> (QLabel*)	显示图片缩略图和文件名
<code>hintLabel_</code> (QLabel*)	显示空闲/识别中/成功/失败状态
<code>reply_</code> (QNetworkReply*)	当前 OCR 请求的回复对象
<code>busy_</code> (bool)	是否繁忙，繁忙时禁用所有交互。
<code>netMgr_</code> (QNetworkAccessManager*)	网络请求管理器

成员方法	说明
<code>startOcr()</code>	构造包含图片 Base64 数据的请求体，发起豆包 OCR 图像识别 API 请求
<code>onOcrFinished()</code>	解析 API 返回的 JSON 结果，提取文字并通过信号（Signal）向外输出

实现逻辑: 上传/拖拽图片 → 缩略图预览 → 图片转 Base64 → 豆包 OCR API → 信号返回识别文本。

MessageBubble 为对话气泡组件，根据 isUser_ 分配样式（用户蓝底右对齐，AI 灰底左对齐），在 AiSolverPage 中用于展示每条消息。

(五) 学习中心模块 `src/modules/learning/`

1. LearningPage：学习中心主入口

成员变量	说明
<code>m_stack</code> (QStackedWidget*)	页面栈容器，管理 Dashboard 与各子页面切换
<code>m_studyTimer</code> (QTimer*)	学习计时器，统计页面停留时长，用于学习管理中心记录学习时长

成员方法	说明
<code>buildDashboard()</code>	构建主面板：四个统计卡片（今日学习/进度/错题数/推荐练习）+ 四个入口卡片（知识点/练习/考试/错题本）
<code>handleStudyTimerTick()</code>	计时器槽函数：累加学习时长，按小时/模块分类存储至 QSettings
<code>showKnowledge()</code> / <code>showPractice()</code> / <code>showExam()</code> / <code>showWrongBook()</code>	导航至对应子页面

2. KnowledgePage：知识库页

成员变量	说明
<code>m_chapterTree</code> (QTreeWidget*)	左侧章节目录树，节点绑定 md 资源路径
<code>m_contentView</code> (LatexTextBrowser*)	右侧内容展示区，支持 LaTeX 渲染

成员方法	说明
<code>buildChapterTree()</code>	构建 10 章高等代数目录树，每个叶子节点存储 qrc 资源路径
<code>loadContentFromResource(path)</code>	读取 md 文件中的 Markdown 内容，渲染成 HTML 并显示在界面中

3. PracticePage：练习页

成员变量	说明
<code>chapterQuestions</code> (QVector)	当前章节目录列表
<code>currentQuestionIndex</code> (int)	当前题目索引
<code>answerWidget</code> (QWidget*)	动态创建的答案交互区

成员方法	说明
<code>generateQuestionByType(type)</code>	随机生成矩阵秩/行列式/特征值/惯性指数/多项式 GCD 五种题型
<code>onSubmitAnswer()</code>	获取用户答案，调用数学引擎比对判题
<code>saveToWrongBook(q)</code>	错题自动入库 JSON 文件

实现细节: 练习入口页以卡片形式展示2种练习模式：计算题、对应章节练习。计算题子页面内置五类随机出题引擎（秩/行列式/特征值/惯性指数/多项式 GCD），基于 `algemate_math` 库生成矩阵数据并计算标准答案，用户输入后比对判题。章节练习子页面左侧为章节目录树，与 KnowledgePage 共享相同章节结构，点击节点通过 `QuestionBank::getQuestionsByChapter()` 加载对应题目。客观题自动判题，主观题调用 DeepSeek API 进行智能批阅。错题自动入库 `wrong_questions.json`。

4. ExamPage：考试页

成员变量	说明
<code>questions</code> (QVector)	当前试卷题目列表
<code>remainingSeconds</code> (int)	剩余考试秒数
<code>m_examTimer</code> (QTimer*)	倒计时器

成员方法	说明
<code>onStartExam()</code>	从 PastExams 获取试卷，初始化进度页
<code>onSubmitExam()</code>	交卷：客观题自动判分，主观题调用 AI 判卷
<code>finishExamAndSave()</code>	汇总成绩，保存至 exam_history.json

5. WrongBookPage：错题本页面

成员变量	说明
<code>contentLayout</code> (QGridLayout*)	2 列网格布局展示错题卡片

成员方法	说明
<code>loadWrongQuestions()</code>	读取 wrong_questions.json，按时间/次数排序渲染
<code>addWrongQuestionCard()</code>	创建错题卡片：题型标签 + LaTeX 题干 + 错误次数 + 查看解析按钮
<code>removeWrongQuestionById()</code>	从 JSON 中移除指定错题
<code>startShuffleRedoWorkflow()</code>	打乱错题顺序，逐个弹出重做对话框

实现细节: 从 `wrong_questions.json` 读取数据，以 2 列网格卡片展示。每张卡片包含LaTeX 渲染的题干、错误次数和"查看解析"按钮，并提供"移出错题本"功能。顶部工具栏支持按时间/次数排序、清空所有错题。"错题重做"可以打乱全部错题顺序，逐题弹出输入对话框，用户作答后自动判题，答对的题目从错题本移出。

6. LearningCenterPage：学习管理中心页面

成员变量	说明
<code>m_lblTomatoCount</code> (QLabel*)	今日番茄钟已专注次数展示标签（动态关联 TomatoManager 实例数据）
<code>m_records</code> (QListWidget*)	自动打卡学习记录列表视图（展示最近30天的打卡时长和模块分布摘要）
<code>m_moduleChart</code> (QWidget*)	今日模块时间分布扇形图微件（ModuleDistributionChart 实例）

成员方法	说明
<code>refreshData()</code>	刷新页面数据总入口：触发学习记录列表刷新与今日模块分布图表的重绘更新
<code>showEvent(QShowEvent*)</code>	重写 QWidget 虚函数：在页面每次切入并显示时，自动调用 refreshData() 保证数据的实时性
<code>backRequested()</code> [Signal]	导航信号：用户点击"← 返回"按钮时向 MainWindow 抛出，用于切回上一级 Dashboard 视图
<code>refreshRecords()</code>	调用全局方法重新读取 QSettings 配置，清空并重新填充 m_records 列表中的打卡历史项

成员方法	说明
<code>updateTomatoRecord()</code>	同步并更新番茄钟历史文本与专注强度的统计信息

7. RecommendPracticePage：推荐练习页面

成员变量	说明
<code>m_internalStack</code> (QStackedWidget*)	页面栈管理器，用于切换批次列表页与具体做题页
<code>m_batchListPage</code> (QWidget*)	推荐历史题集批次列表的容器页面
<code>m_practicePage</code> (QWidget*)	具体题目答题与 AI 批阅的容器页面
<code>m_batchListLayout</code> (QVBoxLayout*)	动态批次列表的垂直布局管理器
<code>m_allBatches</code> (QJsonArray)	缓存本地全部推荐题集的 JSON 数组数据
<code>m_questions</code> (QVector)	当前选中批次下的所有题目结构体数据
<code>m_currentIndex</code> (int)	当前正在作答的题目索引
<code>m_currentBatchIndex</code> (int)	当前练习题集在本地 JSON 中的批次索引
<code>m_questionLabel</code> (QLabel*)	题干文本显示控件（实际指向 LatexTextBrowser）
<code>m_answerWidget</code> (QWidget*)	动态答案交互区域（根据题型生成输入框）
<code>m_feedbackLabel</code> (QLabel*)	判卷反馈结果显示控件（实际指向 LatexTextBrowser）
<code>m_progressLabel</code> (QLabel*)	做题进度状态标签，展示当前题号与总题数

成员方法	说明
<code>reloadQuestions()</code>	读取本地推荐题集配置文件，动态刷新并渲染批次列表
<code>loadBatch(int)</code>	槽函数：解析指定批次的题目数据，并跳转至做题页面
<code>backToList()</code>	槽函数：返回题集列表主页，并自动刷新最新的答题进度
<code>loadQuestion(int)</code>	加载并显示指定索引的题目，重置界面反馈区域状态
<code>updateUIForQuestion(const Question&)</code>	根据题目类型动态构建答题输入框与批阅操作按钮
<code>onAiGradeSubjective()</code>	核心槽函数：调用 DeepSeek API 对主观推导过程进行异步网络请求
<code>markCurrentQuestionCompleted()</code>	更新当前题目的作答状态，并实时同步写回本地 JSON 缓存
<code>saveToWrongBook(const Question&)</code>	判卷未通过时，自动将错题、用户答案及标准答案归档至错题本

实现逻辑: 过内置 `QStackedWidget` 管理两层视图。第一层读取 `recommended_questions.json` 展示历史推荐批次卡片及做题进度；点击卡片后解析对应数据，切换至第二层的具体答题交互界面。还进行了公式渲染与主题联动，题干与 AI 反馈均采用 `LatexTextBrowser` 进行高代数学公式的矢量级渲染。通过 Lambda 表达式监听全局 `themeChanged` 信号，在深浅色模式切换时自动清空缓存并重绘界面，解决 LaTeX 文本在暗色背景下的反色显示问题。引入AI 主观题判阅与错题流转，过程与 `PraciticePage` 部分相同。

（六）数学引擎库 `src/math/`

核心数值类型

类名	功能说明
<code>Matrix<T></code>	模板矩阵类，支持加减乘除、转置、行列式、秩、求逆等线性运算
<code>Polynomial<T></code>	一元多项式，支持四则运算、求导、最大公因式求解
<code>SymbolicExpr</code>	符号表达式，支持变量赋值与多项式化简
<code>Fraction</code> / <code>Complex</code> / <code>BigInt</code> / <code>AlgReal</code>	分数、复数、高精度大整数、代数实数封装类型

核心算法类

类名	功能说明
LinearAlgebra	线性方程组求解、LU 分解、矩阵秩、行列式计算
RealEigen / ComplexEigen	实/复矩阵特征值、特征向量求解
JordanForm	Jordan 标准形求解（含 λ -矩阵、初等因子计算）
SVD	矩阵奇异值分解
Orthogonalization	Gram-Schmidt 施密特正交化
OrthogonalDiag	实对称矩阵正交对角化
BilinearForm	双线性型、二次型标准化、配方法
RationalCanonical	矩阵有理标准形
PolynomialAlg	多项式辗转相除、因式分解

所有算法统一通过 `StepSequence` 存储每一步推导过程，供上层UI渲染分步公式。

LaTeX 渲染引擎 `src/latex/`

类名	功能说明
LatexRenderer	将 LaTeX 公式字符串渲染为 QPixmap 矢量图片（基于 JKQTMathText）
LatexInlineHandler	解析 Markdown 内联公式 <code>\$...\$</code> 、块公式 <code>\$\$...\$\$</code> ，替换为图片标签
LatexTextBrowser	继承 QTextBrowser，重写渲染逻辑自动解析并绘制公式

（七）ui设计模块 `src/ui/`

1. TomatoManager / TomatoClockDialog：番茄钟

成员变量	说明
currentState_ (State)	当前番茄钟所处的运行状态（空闲/专注/短休/长休）
remainingSeconds_ (int)	当前状态阶段下逻辑计数器剩余的秒数
labelStatus_ (QLabel*)	显示当前状态文本的标签（如“专注中/休息中”）
labelTime_ (QLabel*)	高亮渲染的倒计时时间文本标签
btnPauseResume_ (QPushButton*)	控制计时任务“暂停”或“继续”流转的功能按键
btnSkip_ (QPushButton*)	放弃当前计时并复位重置的控制按键
成员方法	说明
instance() [static]	全局获取番茄钟底层管理器单例对象的静态访问入口
setConfig(...)	动态修改并重置专注时长、休息时长与长休循环周期的配置接口
startFocus() / reset()	显式触发番茄钟启动指令 / 强制置空复位并关闭弹窗视图
onTimerTick()	槽函数：响应定时器秒级溢出，执行秒数递减与状态自动流转
buildUi() / initConnections()	构建流线型纵向布局 / 绑定倒计时 <code>tick</code> 与结束信号
updateStyleByStatus(...)	依据最新状态，动态为弹窗切换红（专注）/绿（休息）配色

实现逻辑：

采用经典 **MVC 架构**。核心控制层收拢于单例 `TomatoManager` 中，内部通过 `QTimer` 触发 `onTimerTick()` 递减秒数。当专注时间截止时，自动累加计数并依据循环阈值自动切换状态至短休或长休，向全系统异步辐射状态更迭信号。UI 表现层

`TomatoClockDialog` 弹出时, 通过 Qt 信号槽与全局单例进行异步绑定。弹窗无卡顿地实时重刷时间数字, 并依据状态更迭一键粉刷配色方案, 实现核心数据逻辑与悬浮 UI 的同步协作。

(八) 模块关系总图

```
main.cpp
+-- MainWindow
+-- TitleBar ----- ThemeManager, UserProfile, TomatoManager
+-- NavigationBar ----- 信号转发至 MainWindow 切换页面
+-- QStackedWidget
+-- HomePage ----- UserProfile
+-- CalculatorPage
|   +-- InteractivePage --- Evaluator -> Lexer + Parser + Value
|   +-- DemoPage ----- math/algorithm/*
|   +-- VisualizePage ----- QuadricWidget
+-- AiSolverPage ----- OcrAttachWidget + MessageBubble
+-- LearningPage
|   +-- KnowledgePage --- LatexTextBrowser
|   +-- PracticePage ---- QuestionBank
|   +-- ExamPage ----- QuestionBank + PastExams
|   +-- WrongBookPage --- 本地错题存储模块
|   +-- LearningCenterPage
|   +-- RecommendPracticePage
+-- SettingsPage ----- UserProfile
```

三、小组成员分工情况

- 鲍欣平: 智能解题与学习中心模块 (知识点模式、练习模式、考试模式、错题本)、视频制作
- 任喜悦: 学习管理中心 (学习时长、目标设置等)、设置中心、UI 界面美化设计、icon 设计、番茄钟
- 陈泓宇: 计算助手 (交互计算、算法演示、二次曲面渲染)、LaTeX 渲染、练习模式计算题

四、项目总结与反思

本项目针对高等代数课程学习设计, 整合线性代数数值计算、分步推导、AI 智能答疑、三维几何可视化、题库练习五大功能, 形成一体化桌面学习工具, 覆盖课堂练习、课后刷题、考前模拟三类学习场景。

开发过程中我们遇到并且攻克的难点为: 大模型多模态流式对话、数学图片 OCR 识别、各种数学算法、OpenGL 三维二次曲面渲染、Markdown 与 LaTeX 混合文档渲染、Git 版本控制不规范导致冲突、初步设计不够完善导致后续调整麻烦等。

本次大作业使得我们积累了 Git 版本控制、Qt 桌面程序开发、大模型接口对接、三维可视化渲染的开发经验, 也认识到了项目前期架构设计、边界异常处理的重要性。

以下是我们小组成员各自的总结与反思:

• 鲍欣平:

本项目的最初灵感来源于陈泓宇对计算机实现矩阵运算优化的天才想法。后来我们结合自身学习线性代数的痛点, 把最初想法升级成了一款线性代数学习助手的软件, 这也给我自己的高代期末复习带来了便利。

非常感谢两个超级给力的队友, 给了我非常愉快的合作体验。陈泓宇一开始就设计好了整个代码架构, 我和任喜悦就只需要修改自己负责的部分, 比如我只需要修改 `ai_solver` 和 `learning` 板块; 任喜悦不仅对 UI 的设计有自己的想法, 实现了淡雅美观的界面, 还专门设计了独特的 icon, 展现数学之美。大家从最开始就分工明确, 每个人干自己喜欢的部分, 遇到困难互相帮忙, 加入新功能的时候顺畅, 最终 merge 的时候也几乎没有遇到任何冲突。这让我体会到清晰的代码框架和明确的分工有多么重要。

总结: 对于我自己来说, 我学会了 Qt 网络编程中 API 调用, 学习处理了多模态输入, 如 Base64 编码、OCR 识别; 加深了对 QSettings 数据持久化以及 QTreeWidget 树形导航的理解; 在错题本功能中实践了 JSON 文件的读写与卡片式 UI 的动态构建。

反思：第一次合作项目，git 很多操作还是很方便的，但还有待加强熟练。另外在 ExamPage 中 AI 评阅主观题时一开始没考虑异步任务计数，导致提前收卷，用 `m_pendingAITasks` 计数器跟踪后才解决。

未来展望：当然，现在的版本还有可以完善改进的地方，比如 LaTeX 渲染不够充分，考试模式仅限于固定的往年题，不支持自动组卷。AI 智能解题只是集成了豆包和 DeepSeek，没有实现专属的线性代数智能解题，未来可能会探索 RAG 等方式，让 AI 学习丘维声教材，实现更精准的线性代数专属智能解题。

- 任喜悦：

本项目成功构建了高等代数智能伙伴应用 (AlgeMate)，采用经典 MVC 架构和 Qt 信号槽机制，实现了底层核心逻辑与轻量视图层的异步绑定，确保了多模态 AI 解题、分步交互计算及 3D 曲面绘制等高负载任务的流畅运行。针对高等代数大量 LaTeX 矢量公式的动态渲染需求，系统重点攻克了全局主题 (light / dark) 切换技术；通过引入 Lambda 闭包实时响应深浅色模式变更信号，一键刷新控件样式并强制重绘公式缓存，彻底解决了数学文本在暗色背景下的反色与模糊缺陷，保障了界面全局配色的一致性。

在项目迭代中，我们始终坚持站在真实用户的角度去设计每一项细节，例如为降低公式打字门槛而开发 OCR 图片识别模块、在耗时联网判卷时设计流线型骨架屏占位提示以缓解用户焦虑，充分体现了对用户体验的重视。然而，我在前端 UI 的艺术打磨与视觉交互上投入略少；目前整体界面虽排版流利，但精致度和过渡动画距离极高美观度的商业级应用仍有一定差距，未来将通过更丰富的 QSS 样式定制和动效微件进行视觉升级。

- 陈泓宇：

我在这个项目中学习到了很多神奇的东西，比如多项式复根的数值迭代求解方法 (Durand-Kerner)、有理数域上因式分解的 Zassenhaus 算法等，当然也遇到很多困难，比如一开始妄想能实现代数数矩阵的精确运算，但是发现难度太大程序很容易卡崩，后来就放弃了转为近似求解。关于边界处理和软件测试我也学到了很多，交互计算部分我测试了各种奇怪的输入以保证程序尽可能不崩溃（各种防止死循环、防止数据溢出等），算法演示部分我测试了高代上下两册书所有相关的计算题，以保证准确性。我觉得计算助手最创新的部分应该是算法演示部分，程序计算的时候会记录中途的操作，最后一并输出，用户能看到完整的过程（而且大多是仿照书本例题的格式，对学习比较有帮助）。另外感谢两位队友，我们的小组分工明确，大家也都很靠谱，都做的很好。